

Chapitre 4 : Régression Logistique

Apprentissage Supervisé

Dr Seydou Nourou SYLLA

Université Alioune DIOP de Bambey
Master SI/SR

2 mars 2026

Plan du chapitre

- 1 Du linéaire au probabiliste
- 2 Fonction de coût et descente de gradient
- 3 Implémentation from scratch
- 4 Régularisation

Pourquoi pas une régression linéaire ?

Tentative naïve : modéliser $y \in \{0, 1\}$ par

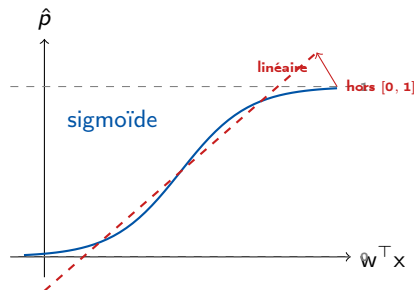
$$\hat{y} = w^T x + b$$

Problèmes :

- $\hat{y}$ peut être < 0 ou > 1 (pas une probabilité !)
- Les résidus ne sont pas gaussiens
- Non robuste aux outliers (distord la droite)

Ce qu'on veut :

$$\hat{p} = P(y = 1 | x) \in [0, 1]$$



La fonction sigmoïde

Solution : Appliquer la fonction sigmoïde (logistique) :

$$\sigma(z) = \frac{1}{1 + e^{-z}} \in (0, 1)$$

Propriétés :

- $\sigma(0) = 0.5$ (point d'inflexion)
- $\lim_{z \rightarrow +\infty} \sigma(z) = 1$
- $\lim_{z \rightarrow -\infty} \sigma(z) = 0$
- $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ (dérivée élégante)

Modèle de régression logistique :

$$P(y = 1 \mid x) = \sigma(w^\top x + b) = \frac{1}{1 + e^{-(w^\top x + b)}}$$

Interprétation : $w^\top x + b$ est le **log-odds** (logit) de la probabilité d'attaque.

Interprétation des poids – Exemple d'application 1

Log-odds :

$$\log \frac{P(y = 1)}{P(y = 0)} = w_1 \cdot \text{duration} + w_2 \cdot \text{src_bytes} + \dots + b$$

Interprétation de w_j :

- $w_j > 0$: augmenter x_j **augmente** la probabilité d'attaque
- $w_j < 0$: augmenter x_j **diminue** la probabilité d'attaque
- $|w_j|$ grand : feature très influente

Exemple :

- $w_{\text{src_bytes}} = -2.3$
 $\Rightarrow$ src_bytes élevé $\rightarrow$ moins suspect
- $w_{\text{logged_in}} = -3.1$
 $\Rightarrow$ connexion authentifiée $\rightarrow$ moins suspect

Règle de décision

$$\hat{y} = \begin{cases} 1 & \text{si } \hat{p} \geq 0.5 \\ 0 & \text{sinon} \end{cases}$$

Mais le seuil 0.5 peut être ajusté selon les besoins (cf. chapitre Évaluation).

Log-vraisemblance (fonction de coût)

Problème : Trouver w et b qui maximisent la vraisemblance des données.

Pour n exemples i.i.d. de loi de Bernoulli :

$$\mathcal{L}(w, b) = \prod_{i=1}^n \hat{p}_i^{y_i} (1 - \hat{p}_i)^{1-y_i}$$

On minimise le **log-loss** (log-vraisemblance négative) :

$$\mathcal{J}(w, b) = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{p}_i) + (1 - y_i) \log(1 - \hat{p}_i)]$$

Propriété

Cette fonction est **convexe** en $(w, b) \Rightarrow$ un seul minimum global. La descente de gradient est garantie de converger.

Gradient et mise à jour (descente de gradient)

Calcul du gradient :

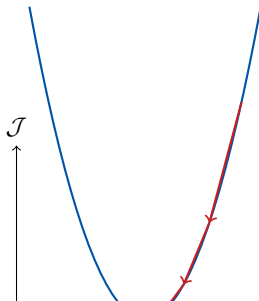
$$\frac{\partial \mathcal{J}}{\partial \mathbf{w}} = \frac{1}{n} \mathbf{X}^\top (\hat{\mathbf{p}} - \mathbf{y}) \quad \frac{\partial \mathcal{J}}{\partial b} = \frac{1}{n} \sum_{i=1}^n (\hat{p}_i - y_i)$$

Mise à jour par descente de gradient :

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \cdot \frac{\partial \mathcal{J}}{\partial \mathbf{w}} \quad b \leftarrow b - \eta \cdot \frac{\partial \mathcal{J}}{\partial b}$$

Variantes de descente de gradient :

- **Batch GD** : gradient sur tout le dataset
- **SGD** : gradient sur 1 exemple (stochastique)
- **Mini-batch GD** : gradient sur un sous-ensemble



Ridge (L2) : $\mathcal{J}_{L2} = \mathcal{J} + \frac{\lambda}{2} \|w\|_2^2$

- Réduit les poids vers 0 (jamais exactement 0)
- Toutes les features utiles

Lasso (L1) : $\mathcal{J}_{L1} = \mathcal{J} + \lambda \|w\|_1$

- Poids exactement 0 $\Rightarrow$ sélection de features
- Beaucoup de features non pertinentes

Dans l'exemple d'application 1

40+ features réseau, moitié non pertinentes $\Rightarrow$
Lasso (L1) préférable pour la sélection
automatique.

Forces et faiblesses de la Régression Logistique

Forces :

- + Produit des **probabilités** (pas juste des classes)
- + Interprétable : poids = influence des features
- + Rapide à entraîner et à prédire
- + Peu de paramètres → robuste au sur-ajustement
- + Baseline solide pour tout problème de classification

Faiblesses :

- Suppose une **frontière linéaire** (limité)
- Moins performante sur les relations non-linéaires
- Nécessite la normalisation
- Sensible aux features corrélées (multicolinéarité)

Pourquoi la LR donne 97% sur l'exemple (vs 99% pour RF) ?

L'exemple d'application 1 a des **frontières de décision non linéaires** (ex : *scan de port* $\equiv$ *src_bytes=0* ET *count* élevé). La régression logistique, étant linéaire, ne capture pas ces interactions.

À retenir

- Modélise $P(y = 1|x) = \sigma(w^T x + b)$
- Optimise le **log-loss** par descente de gradient
- Produit des probabilités calibrées $\Rightarrow$ permet d'ajuster le seuil
- Modèle **linéaire** : bon pour comprendre les données, pas pour les interactions complexes

Hyperparamètre	Description	Recommandé (ex. app. 1)
learning_rate η	Taille du pas	0.01–0.1
n_iterations	Nb d'époques	500–2000
reg_lambda λ	Force régularisation	0.01–1.0
threshold	Seuil de décision	À ajuster (cf. chap. 6)